

# ML Application to Computer Vision

Dr. Bharti Trivedi & Ms.Shweta Rajput

Department of Computer Science & Engineering

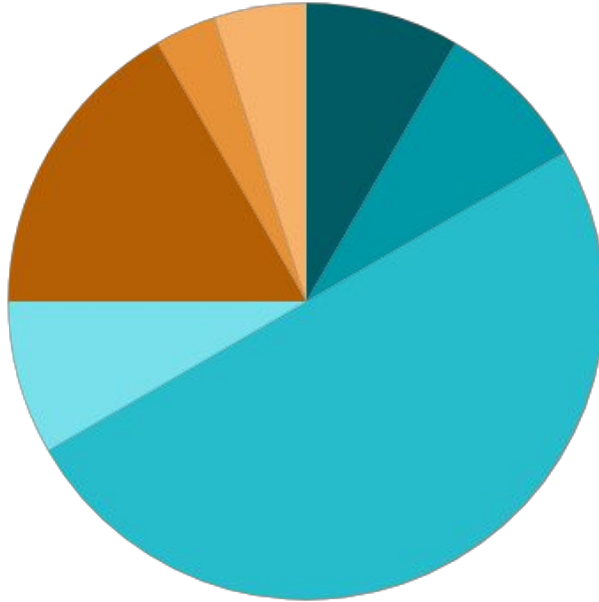


# Outline

1. Human gesture estimation, creating 3D model from 2D images, Computer
2. Vision in Healthcare: Medical Image Analysis,
3. Crop and yield monitoring : Automatic weeding, Insect detection

Source of Image:  
<https://journey.prog-8.com/en/scenes/web-application-development/skills/object-oriented-programming/>

# Lecture Structure



**05 Minutes**

**Outline & Objectives**



**05 Minutes**

**Recap of Previous Lecture**



**30 Minutes**

**Delivery of Actual Content as per Lesson Plan**



**05 Minutes**

**Industry Relevance**



**10 Minutes**

**Interactive Activities**



**02 Minutes**

**Importance of Discipline**



**03 Minutes**

**Recording of Attendance**

Reference: Office Order No. Provost/092023135, Date: 18<sup>th</sup> September 2023



# Objectives

1. Introduction & Background
2. Understand the basic concept of Computer Vision.
3. Understand the basic skills to decide which learning algorithm to use for what problem.
4. Able code up your own learning algorithm and evaluate and debug it.

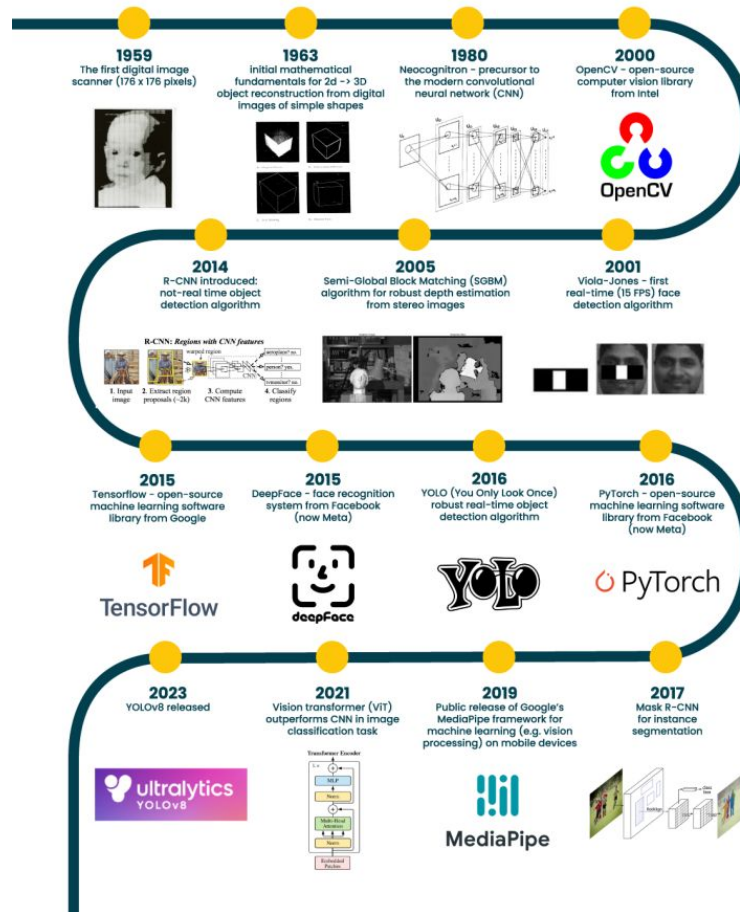


# What is Computer Vision?

- Computer vision is a multidisciplinary field of AI and computer science that focuses on enabling computers to interpret and understand visual information from the world, much like human vision.
- It involves developing algorithms, models, and systems that can process and make sense of images and videos, allowing machines to “see” and extract useful information from visual data.

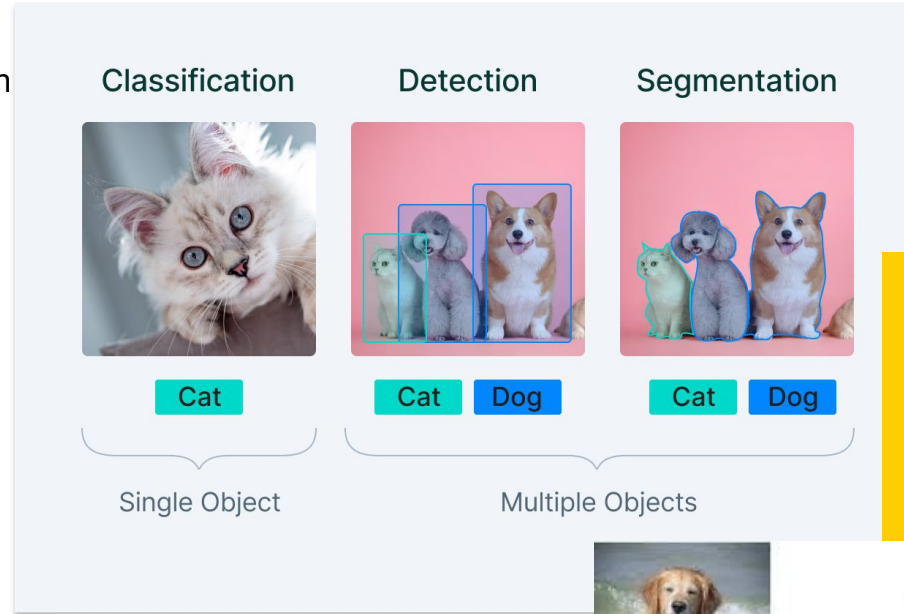
# Evolution of Computer Vision

- **Early Phase:** Rule-based systems (edge detection, segmentation)
- **ML Phase:** Feature extraction + traditional classifiers (SVM, k-NN)
- **Deep Learning Era:** CNNs revolutionized CV by automating feature extraction

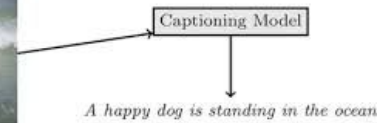


# Core Tasks in Computer Vision

- Image Classification
- Object Detection
- Segmentation
- Pose Estimation
- 3D Reconstruction
- Image Captioning

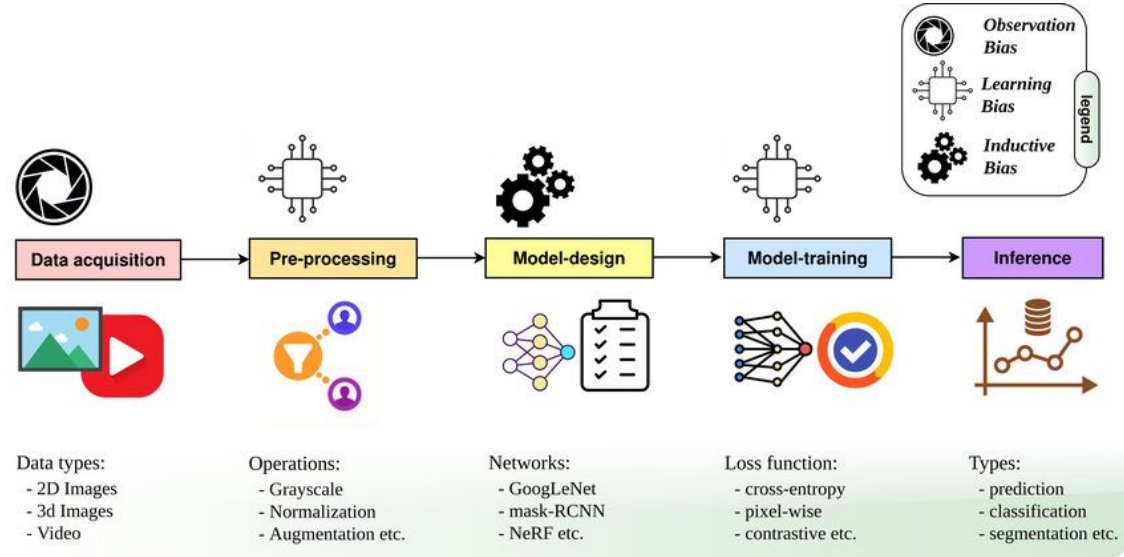


InData Labs



# ML Pipeline in Computer Vision

1. Data Collection
2. Annotation
3. Preprocessing
4. Model Selection
5. Training
6. Evaluation (Accuracy, IoU, mAP)
7. Deployment





# Problem Definition

- **What is the task?**

Classification, detection, segmentation, pose estimation, etc.

- **Why is it important?**

Use-case-driven (e.g., identifying diseases in X-rays, detecting pests in crops)

- **Example:**

“Detect weeds in farm images and classify as harmful or not.”



# Step 1 : Data Collection

- **Sources:**
  - Public datasets (COCO, ImageNet, PlantVillage, etc.)
  - Sensors (CCTV, drones, medical imaging tools)
  - Synthetic generation (GANs, Simulations)
- **Challenges:**
  - Quantity, quality, and variety of images



## Step 2 : Data Annotation

- **Labeling types:**
  - Image-level (e.g., "insect", "weed")
  - Bounding boxes (object detection)
  - Segmentation masks  
(semantic/instance segmentation)
  - Keypoints (pose estimation)
- **Tools:** LabelImg, CVAT, Roboflow, VGG Image Annotator



## Step 3 : Data Preprocessing

- **Cleaning:** Remove corrupt or duplicate images
- **Resizing & Normalization:** Standardize image sizes and pixel values
- **Augmentation:** Improve generalization using:
  - Rotation, flipping, brightness, zoom, noise
- **Splitting dataset:**
  - Training Set (e.g., 70%)
  - Validation Set (15%)
  - Test Set (15%)



## Step 4 : Model Selection

- **Classical ML Models:**
  - SVM, Decision Trees (when features are hand-engineered)
- **Deep Learning Models:**
  - CNN for classification (e.g., VGG16, ResNet)
  - YOLO/SSD for object detection
  - U-Net, Mask R-CNN for segmentation
  - Mediapipe for gesture tracking
- **Transfer Learning:** Use pre-trained models & fine-tune for your task



## Step 5 : Model Training

- **Input:** Images and their labels
- **Goal:** Minimize the loss function (e.g., Cross-Entropy, MSE, IoU Loss)
- **Tools:** TensorFlow, Keras, PyTorch
- **Key Techniques:**
  - Optimizers (SGD, Adam)
  - Epochs & Batching
  - Regularization (Dropout, BatchNorm)



## Step 6 : Model Evaluation

- **Metrics depend on task:**
  - Accuracy, Precision, Recall (Classification)
  - mAP (Object Detection)
  - IoU / Dice Score (Segmentation)
- **Confusion Matrix** to analyze results
- **Visual evaluation:** Overlay predictions on images



## Step 7 : Hyperparameter Tuning

- **Adjust:** Learning rate, number of layers, batch size, dropout rate
- **Methods:**
  - Grid Search
  - Random Search
  - Bayesian Optimization
- **Goal:** Improve performance without overfitting



## Step 8 : Model Testing

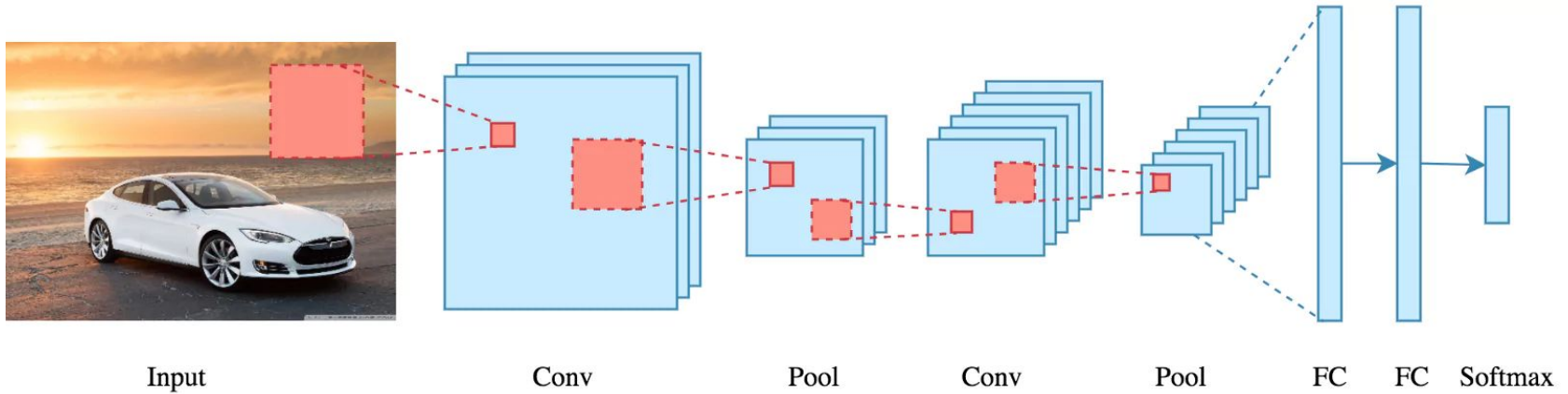
- Use test data only once
- Check generalization ability
- Real-world validation: How does it perform in actual environment?



## Step 9 : Deployment

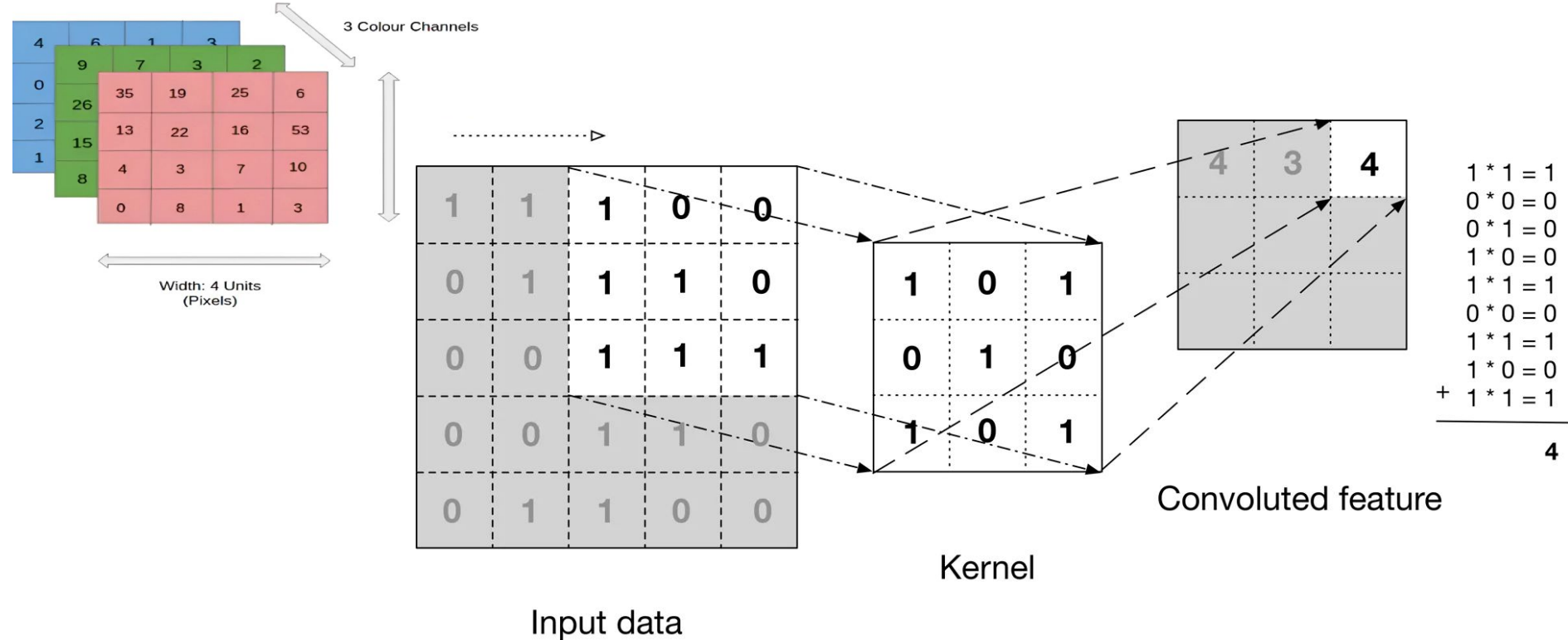
- **Options:**
  - Web/Mobile app (e.g., TensorFlow Lite, ONNX, Flask)
  - Edge Devices (e.g., Raspberry Pi, Jetson Nano)
  - Cloud APIs (e.g., AWS Rekognition, Google Vision API)
- **Concerns:**
  - Latency, Model Size, Inference Speed
  - Model drift and need for periodic retraining

# CNN



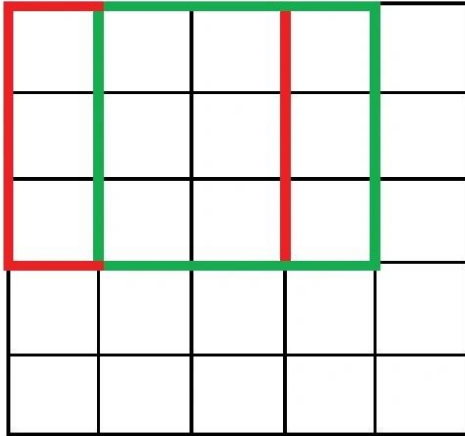
For more Details : [Click Here](#)

# Convolution Layer

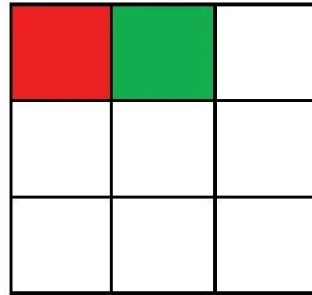


# Convolution Layer

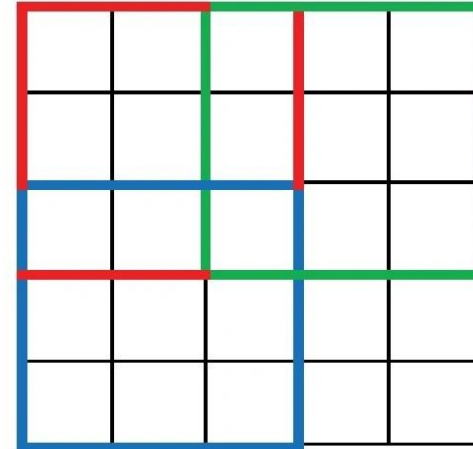
Convolution  
with Stride=1



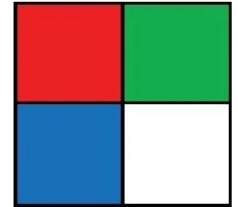
Output



Convolution  
with Stride=2



Output



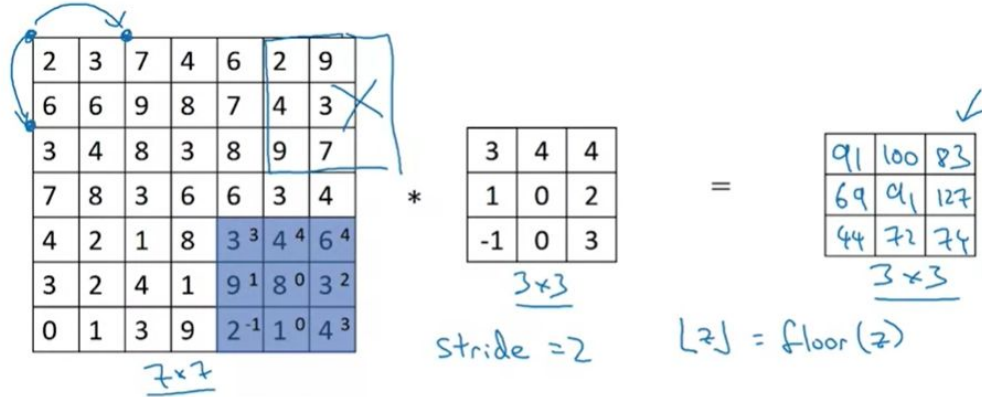
## Summary of convolutions

$n \times n$  image     $f \times f$  filter

padding  $p$     stride  $s$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

## Strided convolution



$n \times n$  \*  $f \times f$   
padding  $p$     stride  $s$   
 $s=2$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

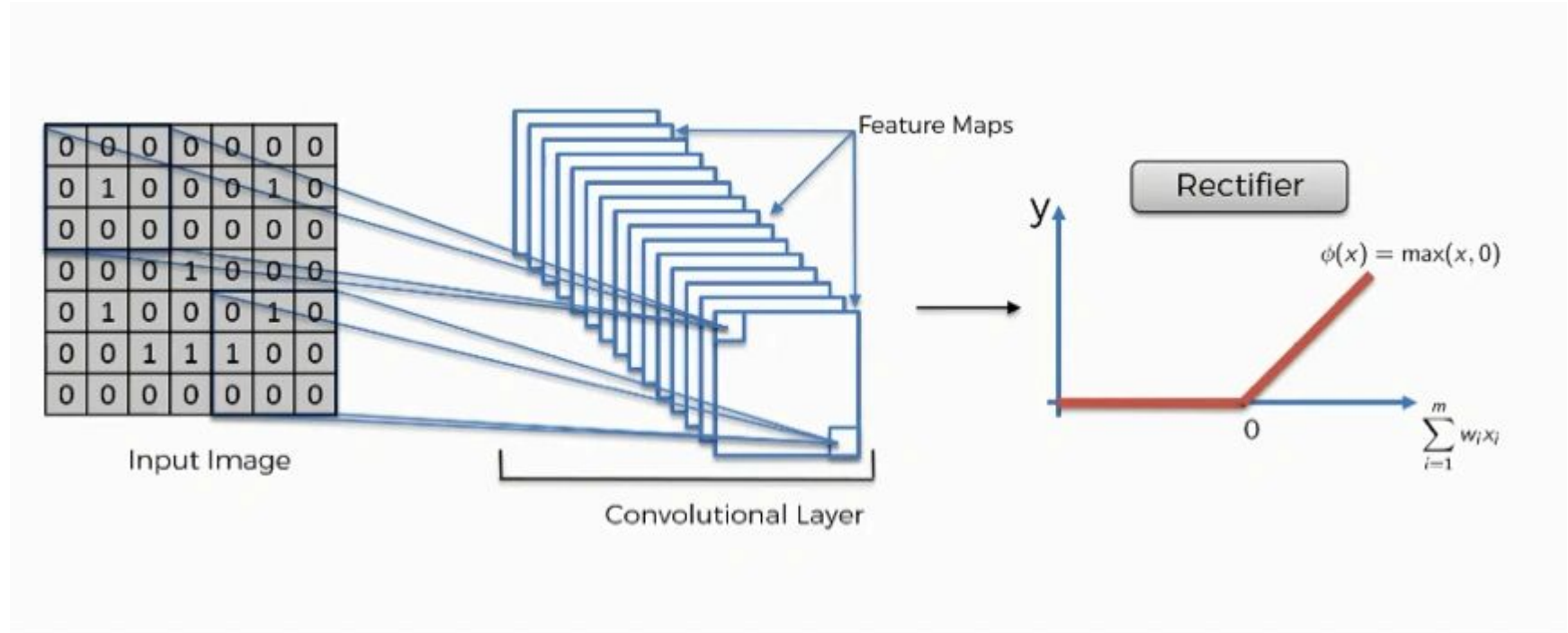
$$\frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$$



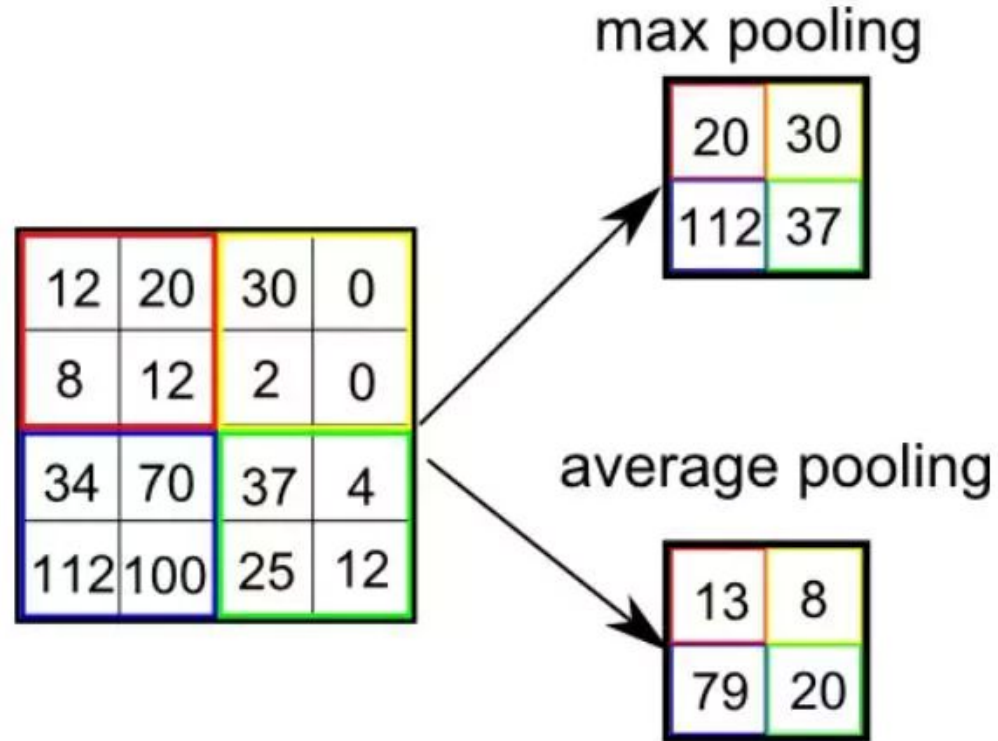
# ReLU

- The rectified linear activation function, or ReLU for short, is a piecewise linear function that, if the input is positive, outputs the input directly; else, it outputs zero.
- Because a model that utilizes it is quicker to train and generally produces higher performance, it has become the default activation function for many types of neural networks.

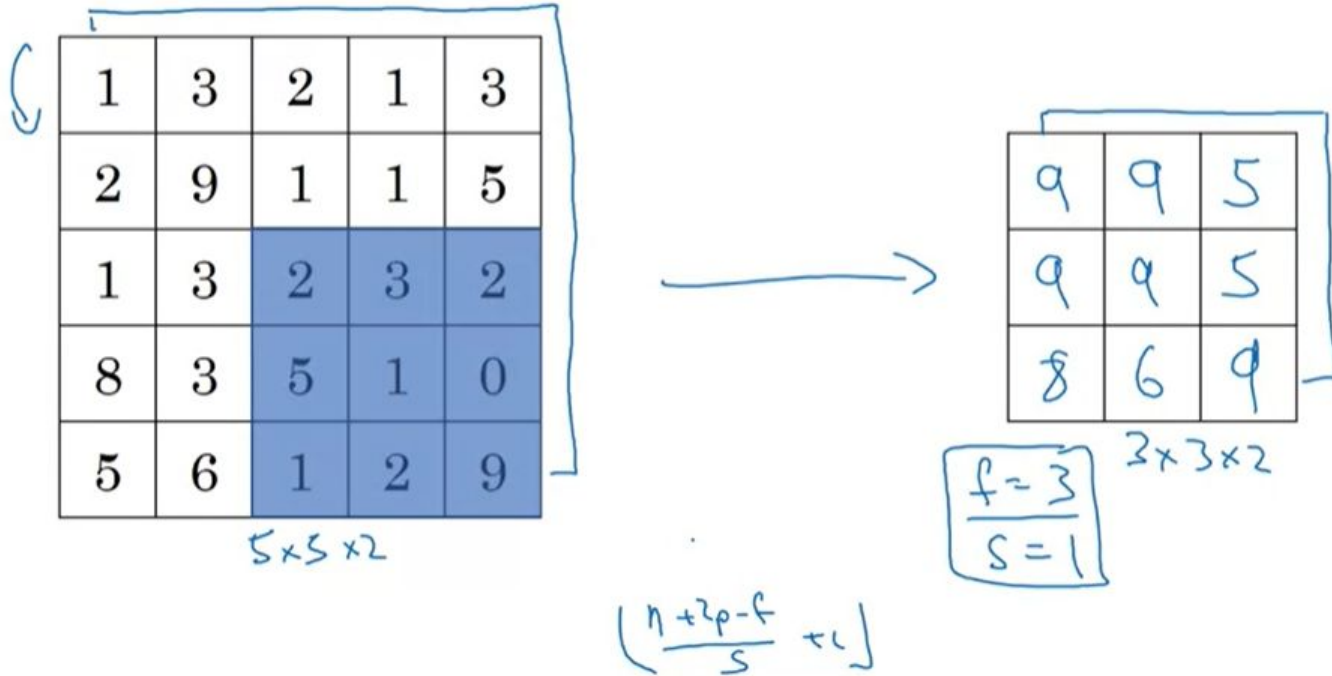
# ReLU



# Pooling Layer

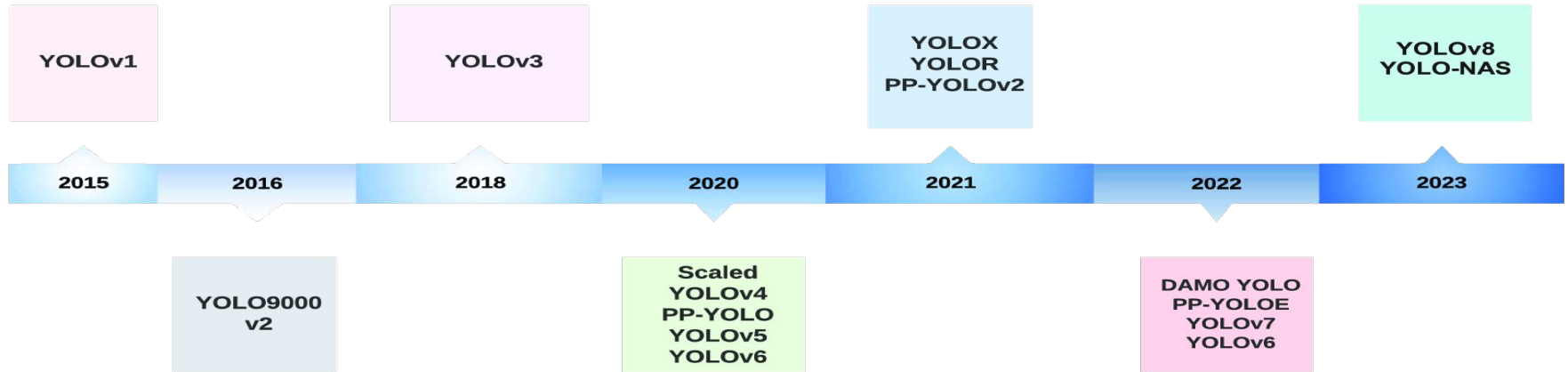


# Pooling layer: Max pooling

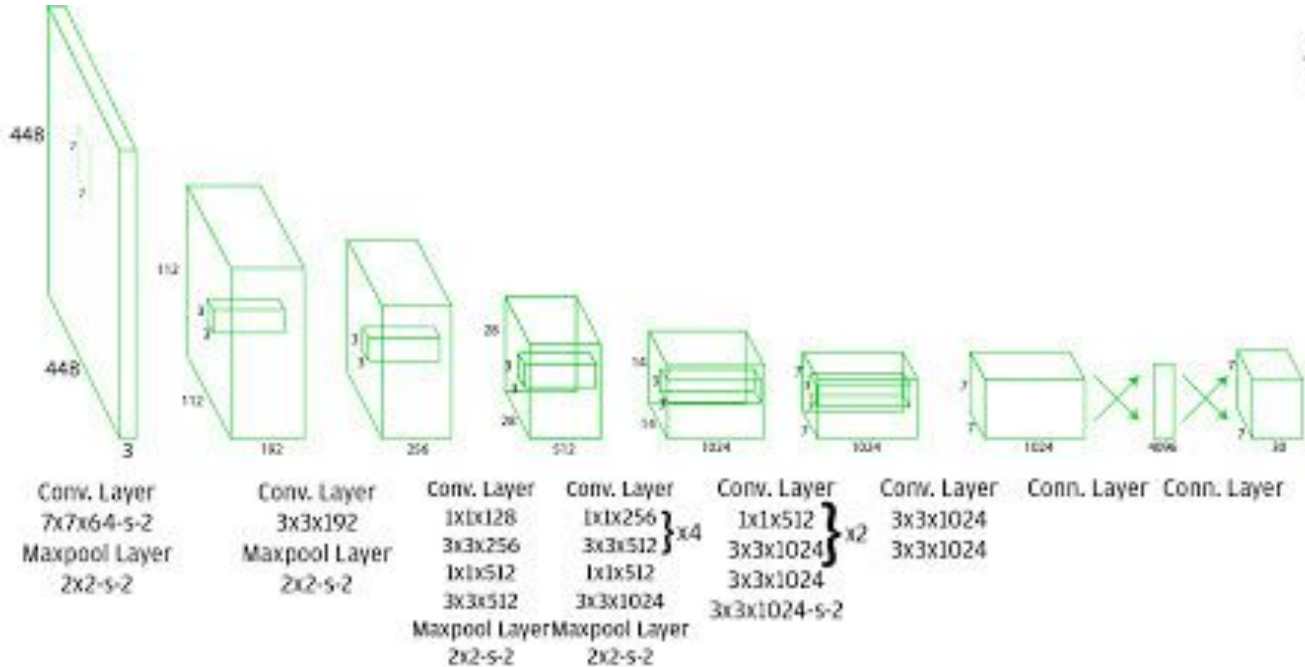


# Object Detection with YOLO

YOLO(You Only Look Once) is a state-of-the-art model to detect objects in an image or a video very precisely and accurately with very high accuracy. In this tutorial, we will learn to run Object Detection with YOLO and plot the frames using OpenCV on both a recorded video and a camera.



# Yolo Architecture





# Yolo Architecture - Explanation

- 1. Input Preprocessing**
- 2. Backbone Convolutional Neural Network (CNN):**
- 3. Use of  $1 \times 1$  and  $3 \times 3$  Convolutions:**
- 4. Fully Connected Layers:**
- 5. Cuboidal Prediction Output:**
- 6. Activation Functions:**
- 7. Output Layer Activation**
- 8. Regularization Techniques - Batch Normalization and Dropout**

## Detection using YOLO

- This architecture divides the image into a grid of  $S \times S$  size.
- If the centre of the bounding box of the object is in that grid, then this grid is responsible for detecting that object.
- Each grid predicts bounding boxes with their confidence score.
- Each confidence score shows how accurate it is that the bounding box predicted contains an object and how precise it predicts the bounding box coordinates with respect to ground truth prediction.





# Human gesture estimation, creating 3D model from 2D images, Computer

- Human gesture estimation, a key area within computer vision and machine learning, is the process by which computers identify, analyze, and interpret human body movements to understand gestures. This field enables more natural human-computer interaction by allowing machines to "see" and understand body language in images and videos.



# Human gesture estimation, creating 3D model from 2D images, Computer

- Gesture estimation is a two-step process:
  - **Pose estimation:** First, the system identifies and localizes specific body parts, like joints, to create a skeletal model.
  - **Gesture recognition:** It then analyzes the sequence of poses to recognize specific actions or gestures, such as waving a hand or signaling a specific word.



# What is Keypoint Detection?

- A **computer vision task** where the algorithm outputs coordinates  $(x,y)$  for each defined body part.
- Example: Detecting **21 hand keypoints** (fingers & joints) or **33 body keypoints** (full pose).

## Approaches to Keypoint Detection : Traditional CV + ML

- Use HOG, SIFT, SURF, Haar features to detect local interest points.
- ML classifiers (SVM, Random Forests) map features  $\rightarrow$  keypoints.
- Limitations: fails in complex backgrounds, not robust to occlusion.



# Deep Learning-Based creating

- Heatmap Regression (Most Common)
  - CNN outputs a heatmap for each keypoint (probability of that joint's location).
  - The peak in the heatmap = keypoint position.
  - Example: OpenPose, HRNet, DeepLabCut.



# 3D model from 2D images



## GSFCU & Values :



 **Thank You..**